Optimising synthetic generators

Alex Davies

February 23, 2026

1 Motivation

Tabular foundation models (e.g. TabPFN) either use synthetic generators based on causal graph priors, or combine a few existing large tabular datasets. Here we'd like to provide a benchmark for the 'realism' of these datasets compared to wild, web-scraped data.

2 Problem

We have two sets of tables, P and Q, real and synthetic respectively. Optimisation requires measuring some distance between them:

$$d = D(P, Q) \tag{1}$$

A good distance measure $D(\cdot)$ should capture:

1. Table-level metrics like row and column count
2. Column-level metrics like type and distribution
3. Inter-column relationships like correlation, causation etc

This is non-trivial, even in DL settings. Models like TabPFN do support data this varied for embeddings, but we risk boot-strapping; we need a method that does not rely on prior learning from synthetic data.

3 Using Graphs (shocker)

Denote a table T, the set of columns C with dimensionality $c_i \in \mathbf{R}^R$, and the set of rows R with dimensionality $r_i \in \mathbf{R}^C$.

Consider some relationship measure $f_{i,j}$ between columns $c_i \in C$ and $c_j \in C$. If this measure properly encodes the mutual information between the two columns, then a complete weighted graph is trivially constructed, such that each edge is encoded through that measure:

$$E = \{e_{i,j} = f_{i,j}(c_i, c_j) \forall i, j \in C\} \tag{2}$$

with node features then

$$X = \{\forall c_i \in C\} \tag{3}$$

.

This formulation has removed no information from the table, yet the graph has removed two heterogeneities preventing comparison from P (node count is flexible; column count is not).

Suppose also some compressive measure over node features (original column values) $\hat{C}_i = g(c_i \in C)$, that aims to preserve distributional properties within c_i. By applying the same measure g across all graphs in P and Q, node feature dimensions are unified.

Node count now preserves the number of columns in the original table. Assuming that f and g properly encode column-level properties, and g interaction-level properties, a graph $G : \{X, E\}$ encodes the input table.

The pipeline, in summary:

1. Compute relationship measure between columns
2. Compute summary statistic over columns
3. Construct graph; node features are from summary statistic, structure from relationship measure

In-practise this may well require thresholding over the relationships measure, to ensure edges encode strong relationships only.

4 Relationship measures over nodes

Our relationships measure r should be:

1. Bounded
2. 'Simple' (not over-parameterised, fast and cheap to compute)
3. Comparable for varying column types (numeric-numeric, numeric-categorical, categorical-categorical)

The ideal candidate here is linear models and R^2/η^2 using absolute values.